

Full Stack Engineer Assignment

Overview

Build a lightweight **Project Management Platform** inspired by applications such as Jira or Trello. The objective of this assignment is to evaluate your ability to design and develop a production-ready application while demonstrating strong software engineering principles, clean architecture, and thoughtful technical decision-making.

The application should showcase your proficiency in backend development, frontend/mobile development, authentication, authorization, database design, containerization, and engineering best practices.

Expected Duration: 2 days

Technology Stack

Backend (Choose One)

- Django
- FastAPI

Client Application (Choose One)

- React.js
- Next.js
- React Native

Database (Choose One)

- PostgreSQL
- MySQL
- SQLite (acceptable for development purposes)

DevOps

- Docker (Mandatory)
 - Docker Compose (Mandatory)
 - GitHub Actions or equivalent CI/CD pipeline (Bonus)
-

Functional Requirements

Authentication

Implement JWT-based authentication.

The application should support:

- User Registration
 - User Login
 - User Logout
 - Protected APIs
 - Secure password hashing
 - Token validation
-

Project Management

Users should be able to:

- Create projects
 - View projects
 - Update projects
 - Delete projects
 - Add project members
 - Remove project members
-

Task Management

Users should be able to:

- Create tasks
- View tasks
- Update tasks
- Delete tasks
- Search tasks
- Filter tasks by status
- Filter tasks by priority
- Paginate task listings

Each task should support appropriate attributes such as title, description, assignee, priority, status, due date, timestamps, and any additional fields you consider necessary.

Suggested task statuses:

- Todo
- In Progress
- Completed

Suggested priorities:

- Low
- Medium
- High

Policy-Based Access Control (PBAC)

Role-Based Access Control (RBAC) must not be used for this assignment.

Authorization should be implemented using a centralized **Policy-Based Access Control (PBAC)** approach. Authorization logic should be separated from business logic and implemented through a dedicated authorization layer.

Example:

```
authorize(user, action, resource)
```

Authorization checks should not be scattered throughout controllers or services.

Your implementation must support the following policies:

Project Visibility

A user can only view projects of which they are a member.

Project Update

Only the project owner may update project details.

Project Delete

Only the project owner may delete a project.

Task Creation

Only project members may create tasks.

Task Update

A task may be updated if:

- The user created the task, or
- The task is assigned to the user.

Task Delete

A task may only be deleted if:

- The task status is **Todo**, and
- The requesting user is the project owner.

Task Completion

A task may only be marked as **Completed** if:

- The task has an assignee.
- All required task information has been completed.

Bonus

Implement configurable policies using JSON, YAML, or another suitable approach instead of hardcoding authorization rules.

Database Design

Design and implement an appropriate relational database schema for the application.

The schema should demonstrate good normalization, meaningful relationships, constraints, and scalability considerations. You are free to decide the database structure based on your application design.

Frontend / Mobile Requirements

Develop the client application using your chosen framework.

The application should include, at a minimum:

- Login
- Registration
- Dashboard
- Project Listing
- Project Details
- Task Details

The dashboard should display useful project statistics such as:

- Total Projects
 - Active Projects
 - Total Tasks
 - Completed Tasks
 - Pending Tasks
-

User Experience

The application should provide:

- Responsive and intuitive user interface (where applicable)
- Form validation
- Loading indicators

- Empty states
- Error states
- Clear user feedback

Task status updates may be implemented using:

- Drag-and-drop interactions (Bonus)

or

- Status selection through a dropdown or similar control
-

API Design

Implement clean and RESTful APIs.

The APIs should demonstrate:

- Proper HTTP status codes
- Input validation
- Consistent response structures
- Error handling
- Pagination
- Filtering
- Searching

Providing Swagger/OpenAPI documentation is encouraged.

Backend Engineering Expectations

The project should demonstrate:

- Clean Architecture
- Modular project structure
- Separation of concerns
- Reusable services
- Centralized exception handling

- Logging
 - Environment-based configuration
 - Maintainable and readable code
-

Testing

Implement at least one of the following:

- Unit Tests
- API Tests

Bonus:

- Integration Tests
-

Containerization

Docker support is mandatory.

The repository should include:

- Dockerfile(s)
- docker-compose.yml

The application should be runnable using a single command such as:

```
docker compose up --build
```

CI/CD (Bonus)

Configure a basic CI/CD pipeline using GitHub Actions or an equivalent solution.

The pipeline may include:

- Dependency installation
- Automated testing
- Application build

- Docker image build
 - Linting (optional)
-

Documentation

Provide a comprehensive README containing:

- Project overview
 - Architecture overview
 - Technology choices
 - Setup instructions
 - Docker instructions
 - Environment variables
 - API documentation
 - Assumptions made
 - Trade-offs considered
-

Git Requirements

Maintain a clean and meaningful Git history.

Example commit progression:

- Initialize project
- Implement authentication
- Implement PBAC authorization
- Add project management APIs
- Add task management APIs
- Develop client application
- Configure Docker
- Add tests
- Update documentation

Avoid submitting the entire project as a single commit.

Bonus Features

Candidates may additionally implement any relevant production-grade features such as:

- Refresh Tokens
 - Email Notifications
 - Activity Timeline
 - File Attachments
 - Redis Caching
 - Background Jobs
 - WebSockets for live task updates
 - Configurable PBAC engine
 - Audit Logging
-

Submission Requirements

Please submit:

- GitHub repository
 - README
 - Postman Collection or Swagger/OpenAPI documentation
 - Docker configuration
 - Sample environment file (`.env.example`)
 - Database migration files
-

General Guidelines

- Use one of the approved technology stacks specified above.
- Write clean, modular, maintainable, and production-oriented code.

- Follow REST API best practices.
- Design the application with scalability and maintainability in mind.
- Use meaningful commit messages throughout development.
- Implement **Policy-Based Access Control (PBAC)**. Role-Based Access Control (RBAC) should not be used.
- Containerize the application using Docker and ensure it can be started with minimal setup using Docker Compose.
- The use of AI-assisted development tools (such as ChatGPT, GitHub Copilot, Claude, Cursor, or similar tools) is **permitted and not restricted**. However, candidates are expected to fully understand every part of their implementation. During the review process, you should be able to explain your code, architectural decisions, design choices, authorization model, and any trade-offs made while building the application. The ability to justify engineering decisions is considered as important as the implementation itself.